



Síntese automatizada de cartas de gestoras

Um pipeline de ponta a ponta, da coleta à entrega em dois canais

Vítor Wilher

Análise Macro

9 de setembro de 2026 | versão 1.0

Resumo. Este artigo documenta a construção de um pipeline automatizado que lê as cartas aos investidores das doze maiores gestoras brasileiras, sintetiza o conteúdo com um modelo de linguagem, gera um exercício replicável em Python e entrega o resultado por e-mail e WhatsApp — sem intervenção humana em nenhuma etapa. O problema central não é técnico no sentido usual: as cartas são públicas e a extração de texto é trivial. A dificuldade está na *heterogeneidade temporal* (dez gestoras mensais, duas irregulares), na *fragilidade das fontes* (doze sites com doze convenções distintas) e, sobretudo, no fato de que **cada etapa automatizada tem um modo de falha silencioso**: código que retorna sucesso sem ter feito nada. Descrevemos as decisões de arquitetura que tornam essas falhas visíveis, os quatro bugs de produção encontrados e corrigidos, e as guardas que impedem o pior desfecho de um sistema de comunicação automática — enviar a mensagem errada para a pessoa certa. O sistema roda semanalmente em produção.

Palavras-chave: automação de dados, modelos de linguagem, CRISP-DM, engenharia de software, gestão de ativos.

As gestoras de recursos publicam periodicamente cartas aos investidores. São documentos públicos, escritos para explicar posicionamento, revisar teses e justificar resultados. Lidas isoladamente, informam sobre uma casa. Lidas em conjunto, informam sobre o mercado: onde o consenso se forma, onde ele racha, e quais premissas cada gestora precisa que sejam verdadeiras para ganhar dinheiro.

Ninguém lê as doze. Não por indisciplina — são centenas de páginas por mês, em periodicidades distintas, e o trabalho de comparar é maior que o de ler. Essa lacuna é o problema de negócio que este projeto ataca.

A contribuição não é o resumo. Resumir texto é a tarefa que modelos de linguagem fazem bem e barato desde 2023. A contribuição é o **pipeline que roda sozinho** e o **par síntese + exercício replicável**: a cada edição, o sistema identifica o mecanismo econômico dominante e gera código em Python que o mede com dado público. O leitor não precisa acreditar na síntese; ele roda o código e discorda com número na mão.

Este artigo documenta as fases do projeto na ordem em que foram construídas, com ênfase deliberada nos erros. As decisões que mais importam no resultado final não vieram de planejamento, mas de falhas observadas em produção.

1 Revisão da literatura

O projeto se apoia em três tradições.

CRISP-DM (Chapman et al. 2000) fornece o esqueleto de processo. A adaptação para projetos com LLM é direta: “modelagem” passa a ser engenharia de prompt e escolha de modelo; “avaliação” torna-se validação qualitativa da síntese contra as cartas originais; “deployment” é o agendamento em nuvem.

Extração de informação de documentos financeiros tem literatura consolidada em relatórios regulatórios — Loughran e McDonald (2011) sobre tom em documentos 10-K é o caso canônico. Cartas de gestoras diferem em um ponto essencial: não são padronizadas. Não há taxonomia, não há campos obrigatórios, e o registro varia do ensaio filosófico ao relatório de atribuição de performance. Isso inviabiliza extração baseada em regras e é o que justifica o uso de um modelo de linguagem.

Engenharia de sistemas automatizados aporta o conceito que mais moldou a arquitetura final: *silent failure*. Nygard (2018) argumenta que sistemas distribuídos falham menos por travamento e mais por degradação silenciosa — componentes que respondem “sucesso” sem ter cumprido a função. Como a Seção 6 mostra, esse foi o padrão de **todos** os bugs sérios encontrados aqui.

2 Metodologia e dados

2.1 As fontes

O universo é a lista das doze maiores gestoras brasileiras. Cada uma publica em seu próprio site, com convenções próprias.

Tabela 1: Periodicidade das fontes

Periodicidade	Gestoras	Exemplos
Mensal	10	Alaska Asset Management, Bahia Asset Management, Occ...
Irregular	2	Dynamo, IP Capital Partners

A heterogeneidade temporal é a decisão central do projeto. Como as cadências diferem, o pipeline **não pode assumir periodicidade única**. Ele roda em cron fixo, detecta o que é novo desde a última execução e sintetiza apenas o delta. Se nada foi publicado na janela, termina sem enviar — silêncio é saída válida, e a Seção 6 mostra o que custou aprender a respeitar essa regra até o fim.

2.2 As estratégias de coleta

Nenhuma das 12 fontes exige login ou navegador headless. Esse achado — verificado site a site — é o que torna o projeto viável com requisições HTTP simples.

Tabela 2: Estratégias de coleta por fonte

Estratégia	Fontes	Quando se aplica
wp_rest	4	API REST do WordPress; contorna JS e HTML pesado
html	3	Links diretos no HTML servido
rss	3	O feed já traz as cartas como itens
url_previsivel	1	URL derivável da data (tratar 404 do mês corrente)
ajax	1	Endpoint admin-ajax.php, sem nonce nem auth

Duas regras emergiram da inspeção e valem registro porque contrariam o instinto:

Nunca construir a URL do PDF por template. Os nomes de arquivo têm sufixos ad-hoc, capitalização variável e acentos. Sempre extrair o atributo `href`.

A data vem do texto da página, nunca do nome do arquivo. Há divergências documentadas: em uma das gestoras, o item rotulado “Julho” aponta para um arquivo chamado **Agosto**. Confiar no nome do arquivo produziria erro de atribuição temporal silencioso — o pior tipo, porque o resultado parece correto.

2.3 Mapeamento ao CRISP-DM

Fase CRISP-DM	Neste projeto
Entendimento do negócio	Lacuna: ninguém lê as doze cartas
Entendimento dos dados	Mapeamento das 12 fontes e suas armadilhas
Preparação	Coleta condicionada ao estado; extração de texto
Modelagem	Prompt que exige mecanismo, não descrição
Avaliação	Validação da síntese contra as cartas originais
Deployment	Cron semanal; entrega em dois canais

3 Implementação

3.1 Fase 1 — Coleta condicionada ao estado

O estado é o campo `ultima_carta` no catálogo, versionado em git e commitado de volta pelo próprio workflow. Sem ele, cada execução reprocessaria o histórico inteiro — desperdício de requisições às fontes e de tokens.

```
# cartas.py - o coração da coleta
for gestora in catalogo["gestoras"]:
    try:
        publicadas = descobrir(gestora)          # estratégia por site
        novas = [c for c in publicadas if e_nova(c, gestora["ultima_carta"])]
    except Exception as erro:
        # Falha por gestora é ISOLADA: um site fora do ar não derruba a edição.
        print(f"[erro] {gestora['nome']}: {erro}", file=sys.stderr)
        continue
```

O `try/except` por gestora não é defensivo por hábito: com doze fontes externas, a probabilidade de todas responderem em uma dada terça é baixa. O isolamento é o que torna o pipeline viável em produção.

3.2 Fase 2 — Síntese com exigência de mecanismo

O prompt de sistema exige, em cada seção, a **cadeia causal** que faz a aposta se pagar — não a descrição do mês. Quando a carta é meramente descritiva, o prompt manda dizer isso explicitamente, em vez de inflar a seção com paráfrase.

Duas guardas cercam a chamada, e ambas nasceram de incidentes reais:

```
if final.stop_reason == "max_tokens":
    raise RuntimeError("A síntese foi cortada por max_tokens...")
texto = "\n".join(b.text for b in final.content if b.type == "text").strip()
if not texto:
    raise RuntimeError(f"A síntese voltou vazia (stop_reason={final.stop_reason})")
```

A segunda guarda existe porque, com esforço de raciocínio no máximo, o modelo consumia o orçamento inteiro de tokens pensando e devolvia texto vazio — **com status de sucesso**. O pipeline escrevia um documento vazio e terminava com código 0. Reduzir o esforço de “máximo” para “alto” resolveu: neste caso, o ajuste mais agressivo produzia menos.

3.3 Fase 3 — O exercício replicável

Dois estágios: um modelo rápido escolhe o conceito dominante entre os que o acervo cobre (88 exercícios indexados); um modelo capaz escreve o exercício novo, com dado atual.

O código gerado **roda em sandbox antes de entrar no documento**. Se falhar, a referência à figura é removida e a edição sai sem o exercício:

```
# A falha do exercício NUNCA derruba a síntese - mesmo princípio
# do isolamento por gestora.
try:
    exercicio = exercicio_da_semana(resumo)
except Exception as erro:
    print(f"[erro] exercício da semana: {erro}", file=sys.stderr)
    exercicio = None
```

3.4 Fase 4 — Entrega em dois canais

E-mail. O broadcast é criado por API, segmentado pela tag exclusiva do projeto, e **agendado** para trinta minutos à frente em vez de disparado no ato — a janela existe para permitir cancelamento.

WhatsApp. Aqui a restrição é de plataforma: a Meta aceita **um webhook por conta**, e ele pertence ao CRM que a equipe comercial usa. Assumir esse webhook mataria o atendimento humano. A solução inverte a direção: em vez de disputar o webhook, o sistema **consome o webhook do próprio CRM**, que notifica quando uma conversa é aberta. O CRM continua dono do canal; nosso código apenas escuta.

Essa inversão só foi encontrada depois de verificar que a alternativa óbvia era impossível: a API do CRM não permite injetar mensagens na conversa nativa (endpoint privado, HTTP 403). O caminho negativo economizou a arquitetura errada.

4 Resultados

Tabela 3: Estado do sistema em produção

Dimensão	Valor
Fontes monitoradas	12
Edições publicadas	4
Exercícios no acervo	88
Testes automatizados	34
Commits no repositório	48

A suíte de testes roda como parte deste artigo: 34 testes, resultado OK. O passo de teste não tolera erro no workflow — uma quebra aborta o job antes de qualquer envio.

4.1 Os quatro bugs que moldaram a arquitetura

Todos compartilham a mesma assinatura: **sucesso aparente, efeito nulo ou errado**. Nenhum apareceu como exceção ou log de erro.

1. Corrupção do arquivo de estado. A rotina que grava o `ultima_carta` usava expressão regular de linha única. Quando o serializador YAML quebrava mapas longos em várias linhas, sobrava um resto órfão que corrompia o catálogo. A correção combina regex multilinha, serialização em linha única e **validação por releitura** antes de gravar.

2. Síntese vazia com status de sucesso. Descrito na Seção 4.2.

3. O filtro de destinatários descartado em silêncio. A versão 3 da API de e-mail aceita o campo que restringe o público, responde HTTP 200 e **descarta o campo**. A releitura devolve nulo. Um envio criado assim iria para a lista inteira em vez do segmento do projeto. A correção foi migrar para a versão 4 da API e, mais importante, **reler o recurso após escrever** para provar que gravou.

4. Reenvio da edição anterior. O mais grave, encontrado em auditoria antes de causar dano. Quando nenhuma gestora publica, o coletor termina com código 0 sem gerar edição. Mas o script de e-mail lê a edição *mais recente do acervo* — que é versionado, logo sempre existe. Numa semana silenciosa, o sistema reenviaria a edição anterior, com o mesmo assunto, aos mesmos leads.

A correção é uma guarda de frescor que recusa edições mais velhas que dois dias, terminando com código 0: “*nada a enviar*” é resultado normal, não falha. Uma segunda guarda impede duplicatas verificando se já existe envio pendente com o mesmo assunto — adicionada depois de um teste agendar dois e-mails idênticos.

4.2 A lição transversal

Os quatro bugs sugerem um princípio operacional:

Em sistemas automatizados de comunicação, **o código de status não é prova de efeito**. Toda escrita que importa deve ser seguida de uma leitura que a confirme.

Esse princípio está implementado em três pontos: validação do YAML após gravar, releitura do broadcast para conferir segmento e agendamento, e verificação do `stop_reason` da síntese. É também o que o passo de resumo do workflow endereça: um passo tolerante a erro que falha deixa a execução verde, tornando “não enviado” visualmente idêntico a “enviado”.

5 Próximos passos

Validação quantitativa da síntese. Hoje a avaliação é qualitativa. O caminho natural é um conjunto de cartas anotadas manualmente com as teses esperadas, e medir cobertura e precisão da extração de mecanismo.

Detecção de contradição entre gestoras. A seção de convergências é gerada pelo modelo em texto livre. Estruturá-la — par de gestoras, dimensão, direção da divergência — permitiria acompanhar a evolução do consenso ao longo do tempo.

Cobertura das séries paralelas. Duas gestoras publicam cartas de crédito além da macro, e uma publica por fundo. Hoje o pipeline cobre a série principal.

6 Conclusão

O sistema descrito roda semanalmente sem intervenção. A parte tecnicamente interessante não foi a síntese — essa é a commodity do momento — mas o conjunto de decisões que tornam a automação *confiável o suficiente para não ser supervisionada*: isolamento de falha por fonte, estado versionado, guardas de frescor e duplicata, e a disciplina de reler toda escrita que importa.

O padrão comum aos quatro bugs de produção é instrutivo para qualquer projeto de dados automatizado. Nenhum deles apareceu como erro. Todos apareceram como sucesso — e foi precisamente por isso que sobreviveram até a produção. A robustez de um pipeline não está em tratar exceções, mas em **desconfiar dos sucessos**.

7 Referências

- Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-step data mining guide*. The CRISP-DM Consortium.
- Loughran, Tim, e Bill McDonald. 2011. “When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks”. *The Journal of Finance* 66 (1): 35–65. <https://doi.org/10.1111/j.1540-6261.2010.01625.x>.
- Nygard, Michael T. 2018. *Release It! Design and Deploy Production-Ready Software*. 2º ed. Pragmatic Bookshelf.